

Lecture 4a: Neural Operator methods

Chris Budd and Aengus Roberts¹

¹University of Bath and KE Hub

Ai4Sci, December 2025

Some papers/books to look at

- Courant and Hilbert *Methods of mathematical physics Volumes 1,2*
- Stuart et. al. *Fourier Neural Operator for PDEs*
- Kovachi et. al. *Operator learning: algorithms and analysis*
- Boullé and Townsend *Learning elliptic PDEs*
- Halko et. al. *Finding structure with randomness*

Motivation: Solution Operators

Have studied using PINNS and DRMs to solve PDE problems of the form

$$u_t = F(\mathbf{x}, u, \nabla u, \nabla^2 u) \quad \text{with BC,}$$

$$u(0, x) = u_0(x)$$

At time T we have the solution $u_T(x) \equiv u(x, T)$.

Solution $u(x, t)$ for all x and t is obtained by minimising a function directly associated with the PDE eg. residual.

Neural Operator methods take a different approach

- Consider u_T as a function F of u_0 . $u_T = F(u_0)$
- F is an operator mapping one infinite dimensional function space to another $F : A \rightarrow B$. eg. $A, B = H^1(\Omega)$
- Train a Neural Operator NN to approximate this operator note infinite dimensions
- Train it by generating a (large) set of solution pairs (u_0^i, u_T^i)

Can generate solution pairs using a (conventional) numerical method eg. Finite Element, Pseudo-Spectral, Symplectic.

eg. ERA5 data for 24 hour weather forecasts.

Example 1: Blind channel identification

- Have an $n \times n$ matrix B
- Have a **random set** of N vectors $\mathbf{x}_i \in R^n$
- Compute the N (noisy) matrix vector products

$$B \mathbf{x}_i = \mathbf{y}_i \in R^n$$

Challenge: Construct the matrix B from the set of N **solution pairs** $(\mathbf{x}_i, \mathbf{y}_i)$

Methodology:

- Let $X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, $Y = [\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N]$
- **Find** $B = Y X^+$ using the **SVD/Moore-Penrose pseudo-inverse**.

Example 2: A linear ODE system

Consider the linear ODE

$$\frac{d\mathbf{u}}{dt} = A \mathbf{u}, \quad \mathbf{u}(0) = \mathbf{u}_0, \quad \mathbf{u} \in \mathbb{R}^n.$$

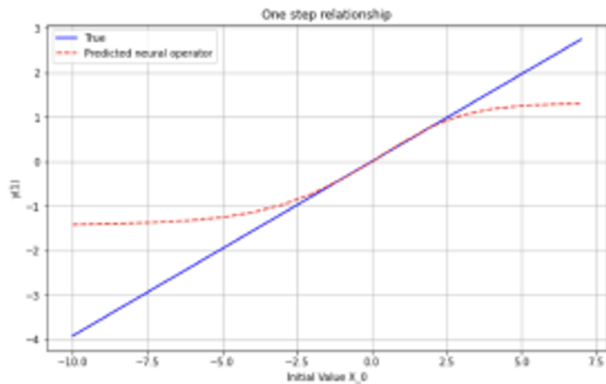
Solution

$$\mathbf{u}(T) = e^{A T} \mathbf{u}_0 \equiv B \mathbf{u}_0$$

$$B \equiv e^{AT} = I + AT + \frac{A^2 T^2}{2!} + \frac{A^3 T^3}{3!} + \dots$$

Properties of the solution operator

- Operator B is linear
- Operator is continuous over any subset of R^n
- Can easily learn the matrix B from data pairs if we assume that the operator is linear in advance!
- If we learn B from a subset of the data pairs then we can extrapolate this to ALL data pairs
- This is NOT true if don't make the linearity assumption. Many NN methods will locally approximate the operator to be linear, but will not give this as a global approximation.



Example 3: Parabolic PDEs

Consider the **parabolic PDE** [picture]

$$u_t = u_{xx} + f(x), \quad x \in [0, 2\pi], \quad u(0, x) = u_0(x), \quad \text{periodic BC}$$

We can express $u(x, t)$ in terms of **convolutional integral operators**:

$$u(x, t) = G * u_0 + H * f \equiv \int_0^{2\pi} G(x-y, t) u_0(y) dy + \int_0^{2\pi} H(x-y, t) f(y) dy$$

These operators act on the **infinite dimensional space** $L^2[0, 2\pi]$.

Can find $G(z, t)$ and $H(z, t)$ **explicitly** using a **Fourier series**.

This methodology motivates the construction of the **Fourier Neural Operator (FNO)**

The FNO: In general

The **FNO architecture** is based on the process of solving the linear heat equation, but also works for **nonlinear problems**. The FNO constructs a 'Neural Map' Ψ parametrised by θ as follows:

$$\Psi(a, \theta)_{FNO} \equiv Q \circ \mathcal{L}_L \circ \dots \circ \mathcal{L}_2 \circ \mathcal{L}_1 \circ P(a).$$

$$\mathcal{L}_n(v)(x, \theta) = \sigma(W_n v(x) + b_n + K(v))$$

Here W is a **pointwise linear local map**. $K(v)$ is a **global convolutional integral operator**, kernel $G_n(\theta)$. Evaluate Kv using an **FFT** via

$$FFT(Kv) = FFT(G_n) FFT(v).$$

FFT restricted to M modes. Nonlinearity and **higher order modes** introduced via the **activation function** σ .

Figure from FNO paper

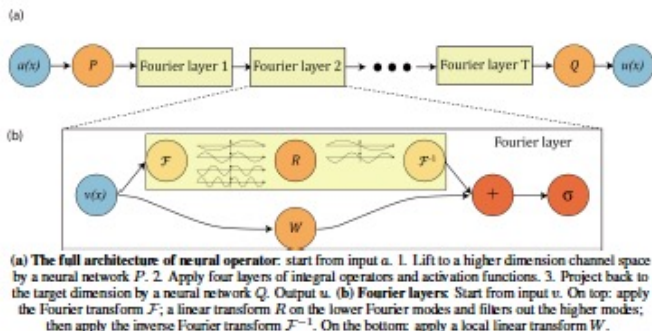


Figure 2: **top**: The architecture of the neural operators; **bottom**: Fourier layer.

FNO in detail

- **Input** $a_j(x) \in \mathcal{A}$ **output** $u_j(x) = N(a_j) \in \mathcal{U}$ are **functions** on $x \in D \subset \mathbb{R}^d$
- Assume have access to pointwise observations of a only at points in $x_i \in D_j \subset D$. **Output u does not depend on D_j : super-resolution**
- **Lift** a to a higher dimensional representation $v_0(x) = P(a(x))$ by a shallow NN.
- Calculate a series of updates $v_n \rightarrow v_{n+1}$ via the local W_n and global (integral) K_n operators:

$$v_{n+1}(x) = \sigma(W_n v_n(x) + (K_n(\theta) v_n))(x).$$

- For example $\sigma = \text{ReLU}$: This introduces **nonlinearity** into the map in a slightly uncontrolled way
- **Project** $v_L \rightarrow u(x) = Q(v_L)$
- **Learn** P, Q, W_n, K_n from the data pairs

Training

- Assume input $a \in \mathcal{A}$
- In the original FNO paper take a_j as an i.i.d sequence from $\mathcal{A}$.
- Construct pairs $(a_j, N(a_j))$ using an accurate solver eg. pseudo-spectral method
- In FNO paper take $N = 1000$ training and 200 training instances. Adam optimiser to find parameters θ via:

$$\min_{\theta} E_{a \sim \mu} [\|\psi(a, \theta) - N(a)\|]$$

- Can significantly improve training by a more careful selection of input and output pairs [Liu, B, et. al.]

FNO online 1

The screenshot shows a web browser displaying the GitHub repository for **NeuralOperator**. The repository is titled "NeuralOperator: Learning in Infinite Dimensions". The main content area includes a description of the library, its installation instructions, and a quickstart section. The right sidebar shows the repository's statistics, including 31 contributors, 211 deployments, and 100.0% Python code coverage.

NeuralOperator: Learning in Infinite Dimensions

`neuraloperator` is a comprehensive library for learning neural operators in PyTorch. It is the official implementation for Fourier Neural Operators and Tensorized Neural Operators.

Unlike regular neural networks, neural operators enable learning mapping between function spaces, and this library provides all of the tools to do so on your own data.

Neural operators are also resolution invariant, so your trained operator can be applied on data of any resolution.

Installation

Just clone the repository and install locally (in editable mode so changes in the code are immediately reflected without having to reinstall):

```
git clone https://github.com/NeuralOperator/neuraloperator
cd neuraloperator
pip install -e .
pip install -r requirements.txt
```

You can also just pip install the most recent stable release of the library on [PyPI](#):

```
pip install neuraloperator
```

Quickstart

After you've installed the library, you can start training operators seamlessly:

Contributors 31

[+ 17 contributors](#)

Deployments 211

- github-pages 9 months ago
- testpypi
- pypi 5 months ago

[+ 208 deployments](#)

Languages

- Python 100.0%

FNO online 2

The screenshot shows a web browser displaying the NeuralOperator GitHub page. The page has a navigation bar with links: **NeuralOperator**, **Install**, **User Guide**, **API**, **Examples**, and **Developer's Guide**. A search bar is located on the left side of the page. The main content area is titled "Examples" and contains a gallery of interactive examples. The first example is "Data", which shows a gallery of interactive examples that showcase how the tools we provide in **neuraloperator** can be applied to a variety of problems. Check out the [User Guide](#) for more detailed information on the theory behind neural operators. The second example is "Layers", which shows examples of individual layers which comprise operators or parts of operators for composition into end-to-end models. The page also includes a sidebar with a menu and a table of contents on the right side.

Search the doc.

Installing NeuralOperator
User Guide
API reference
Examples

- Data
- Layers
- Losses
- Models
- Training and Meta-Algorithms

NeuralOperator Developer's Guide

Data

A gallery of interactive examples that showcase how the tools we provide in **neuraloperator** can be applied to a variety of problems. Check out the [User Guide](#) for more detailed information on the theory behind neural operators.

Examples of NO layers in action.

A simple Darcy-Flow dataset

A simple Darcy-Flow spectrum analysis

Layers

Examples of individual layers which comprise operators or parts of operators for composition into end-to-end models.

ON THIS PAGE

- Examples
- Data
- Layers
- Losses
- Models
- Training and Meta-Algorithm

FNO online 3

The screenshot shows a web browser displaying the NeuralOperator GitHub User Guide. The browser's address bar shows the URL `https://neuraloperator.github.io/dev/user_guide/index.html`. The page has a dark sidebar on the left with a search bar and a menu. The main content area is titled "User Guide" and contains an introduction to the library, a section on "Intro to operator learning", and a table titled "NeuralOperator library structure". The table lists four modules: `neuralop`, `neuralop.models`, `neuralop.layers`, and `neuralop.data`, each with a brief description. The sidebar menu includes links to "Installing NeuralOperator", "User Guide", "API reference", "Examples", and "NeuralOperator Developer's Guide". The "User Guide" section is currently selected.

Firefox File Edit View History Bookmarks Tools Window Help

Search the doc Go

Installing NeuralOperator

User Guide

Intro to operator learning

NeuralOperator library structure

Interactive examples with code

API reference

Examples

NeuralOperator Developer's Guide

User Guide

NeuralOperator provides all the tools you need to easily use, build and train neural operators for your own applications and learn mapping between function spaces, in PyTorch.

Intro to operator learning

To get a better feel for the theory behind our neural operator models, see [Neural Operators: an introduction](#). Once you're comfortable with the concept of operator learning, check out specific details of our Fourier Neural Operator (FNO) in [Fourier Neural Operators](#). Finally, to learn more about the model training utilities we provide, check out [Training neural operator models](#).

NeuralOperator library structure

Here are the main components of the library:

Module	Description
<code>neuralop</code>	Main library
<code>neuralop.models</code>	Full ready-to-use neural operators
<code>neuralop.layers</code>	Individual layers to build neural operators
<code>neuralop.data</code>	Convenience PyTorch data loaders for PDE datasets

ON THIS PAGE

User Guide

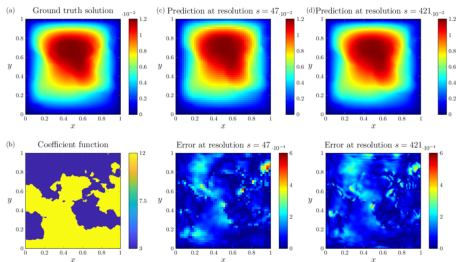
- Intro to operator learning
- NeuralOperator library structure
- Interactive examples with code

Example 4: The Darcy Problem

The **Darcy problem** relates a permeability $a(x)$ to a velocity field $u(x)$

$$-\nabla \cdot (a(x) \nabla u) = f(x) \quad x \in \Omega, \quad u = 0 \quad x \in \partial\Omega.$$

This induces a (nonlinear) map $N : a \rightarrow u$, $N : L^2(\Omega) \rightarrow H_0^1(\Omega)$



We can approximate this map using the **Finite Element Method**

$$u(x) \approx U(x) = \sum U_i \phi_i(x).$$

$$-\nabla \cdot (a(x) \nabla u) = f \implies \int a(x) \nabla u(x) \cdot \nabla \phi_i(x) dx = \int f(x) \phi_i(x) dx \equiv f_i$$

Giving the **linear system**

$$\mathbf{AU} = \mathbf{f}, \quad \mathbf{U}_i = U_i, \quad \mathbf{f}_i = f_i, \quad A_{ij} = \int a(x) \nabla \phi_i \cdot \nabla \phi_j dx.$$

Hence we can approximate the nonlinear map via:

$$\mathbf{U} = \mathbf{A}^{-1} \mathbf{f}$$

And ...

We can **LEARN** this map by

- Doing lots of finite element calculations to find solution pairs $(a(x), u(x))$
- Learn the operator between these pairs using an FNO

Methods such as FNO work as much as possible in the infinite-dimensional function space.

They Construct and train Neural Operators which are **approximations to the true operator (or its inverse)** which are **independent of the resolution of the underlying function/image**

Convergence [Kovachi et. al.]

- FNO and DeepONet can approximate a wide variety of operators
- Assume that input space $\mathcal{U}$ is a separable Banach space and the map N is compact
- Prove convergence on any finite dimensional set using the universal approximation theorem
- Take an appropriate limit (approximation theory of Banach spaces which applies to the sets over which PDEs are typically formulated)

Nonlinear problems and a warning

Consider now the nonlinear parabolic PDE

$$u_t = u_{xx} + f(x, u), \quad u(0) = u(1) \quad u(0, x) = u_0(x)$$

This does not always induce a continuous map from $u(0, x) \rightarrow u(1, x)$.

- If $f(x, u)$ is Globally Lipschitz in x and u then all is OK
- If not then we may have problems
- See Case Study!

Example

Let

$$f(x, u) = u^2, \quad u_0(x) = \gamma > 0$$

Then

$$u(1, x) = \frac{\gamma}{1 - \gamma}.$$

Map is only continuous on the interval $\gamma \in [0, 1)$

If we train only on data with $\gamma < 1$ we will get a false result if we try to extend to $\gamma > 1$.

Areas for improvement and research on FNO

- Observe poor conservation laws at the moment
- Generating a **good training set** is crucial and can be **slow**. How to make it good and fast?
- FNO struggles away from the training set. This is OK for MCMC emulators for UQ.
- BUT Need to broaden its scope and extend the theorems on its convergence
- NONE of this theory applies, for example, to the **nonlinear** heat equation

$$u_t = u_{xx} + u^2.$$